



Lab Project Report

Only for course Teacher						
		Needs Improvement	Developing	Sufficient	Above Average	Total Mark
Allocate mark & Percentage		25%	50%	75%	100%	
Understanding/Analysis						
Implementation						
Accuracy						
Task Efficiency						
Total obtained mark						
Comments						

Semester: Spring / Fall

Student Name: Sandria Alam Samanta

Student ID: 26010173

Batch: 46

Section: B

Course Code: CSE 124

Course Name: Object Oriented Programming

Course Teacher Name: Debabarata Mallick

Course Teacher Designation : Lecturer Submission Date : 13\ 09\ 2026

1.0 Table Of Content

Content	Page
1.0 Table Of Content.....	2
1.1 List Of Figure.....	3
2.0 Project Title : Contact Management System using OOP.....	4
2.1 Problem Statement.....	4
2.2 Project Proposal.....	4
2.2.1 Purpose of the Project.....	4
2.2.2 Goals of the Project.....	4
2.2.3 Scope of the Project.....	5
2.2.4 Methodology.....	5
2.3 Implementation.....	6
2.3.1 UML Class Diagram.....	6
2.3.2 Class Order List.....	7
2.3.3 Class Description.....	7

2.3.4 Flowchart of ProgramExecution.....	8
2.3.5 Dynamic Output Showcase with Code Insights.....	9
2.3.6 Executing Flowchart	12
2.4 Conclusion	12
2.4.1 Remaining work.....	12
2.4.2 Future work	13
2.5 Reference.....	14
2.6. Appendix	14
2.6.1 Github Link	14
2.6.2 Code Snippets.....	15

1.1 List Of Figure :

Figure	Description	Page
Figure 2.1	UML Class Diagram of Contact Management System.....	6
Figure 2.2	Flowchart of Program Execution.....	8
Figure 2.3	User Registration & Login Output.....	9
Figure 2.4	Adding New Contact Output.....	10
Figure 2.5	View All Contacts Output.....	11
Figure 2.6	Search Contact Output.....	11
Figure 2.7	Update Contact Output.....	12
Figure 2.8	Delete Contact Output.....	12
Figure 2.9	Contact Class Definition (Code Snippet).....	15

Figure	File Handling Code (Reading & Writing	16
2.10	Data).....	

2.0 Project Title : Contact Management System using OOP

2.1 Problem Statement:

People often struggle to manage a large number of contacts (friends, family, colleagues, clients) efficiently. Using paper notebooks or phone memory leads to lost numbers, duplicate entries, and difficulty in searching or updating information quickly. There is a need for a lightweight, reliable, and persistent desktop application that allows users to organize, search, update, and store contact information systematically with data persistence.

2.2 Project Proposal:

This project aims to develop a desktop-based console application in Java that allows users to manage their personal or professional contacts, perform CRUD operations (Create, Read, Update, Delete), search contacts, and ensure data persistence using File Handling.

2.2.1 Purpose of the Project:

The primary purpose is to apply Object-Oriented Programming (OOP) principles in Java to build a functional Contact Management System that stores, retrieves, updates, and deletes contact information cleanly and efficiently with secure user access.

2.2.2 Goals of the Project:

- Apply Java OOP concepts like Encapsulation, Inheritance, Polymorphism, Abstraction, and Data Persistence in a practical scenario.
- Guarantee secure access using user registration and login management.
- Provide an intuitive console interface to add, view, search, update, and delete contacts.

- Prevent data loss between sessions using local file storage.
- Implement proper exception handling for invalid inputs.

2.2.3 Scope of the Project:

- **User Authentication:** Login and Registration for secure personal access (Mandatory).
- **Contact Management:** Create, view, search, update, and delete contacts.
- **Search Functionality:** Search contacts by name or phone number.
- **Data Persistence:** Store and load user data dynamically to/from flat text files using File Handling.
- **Error Handling:** Robust exception handling to manage invalid inputs.
- **OOP Principles:** Encapsulation, Inheritance, Polymorphism, Abstraction, Method Overloading & Overriding.

2.2.4 Methodology:

- **Object-Oriented Design:** Structuring data into modular entities (User, Contact, DataStorage).
- **File I/O Integration:** Using Java FileWriter / BufferedReader or Serialization to keep records permanently.
- **Modular Execution:** Separating user interface operations from system business logic.
- **Exception Handling:** Using try-catch blocks for robust error management.

2.3 Implementation

2.3.1 UML Class Diagram:

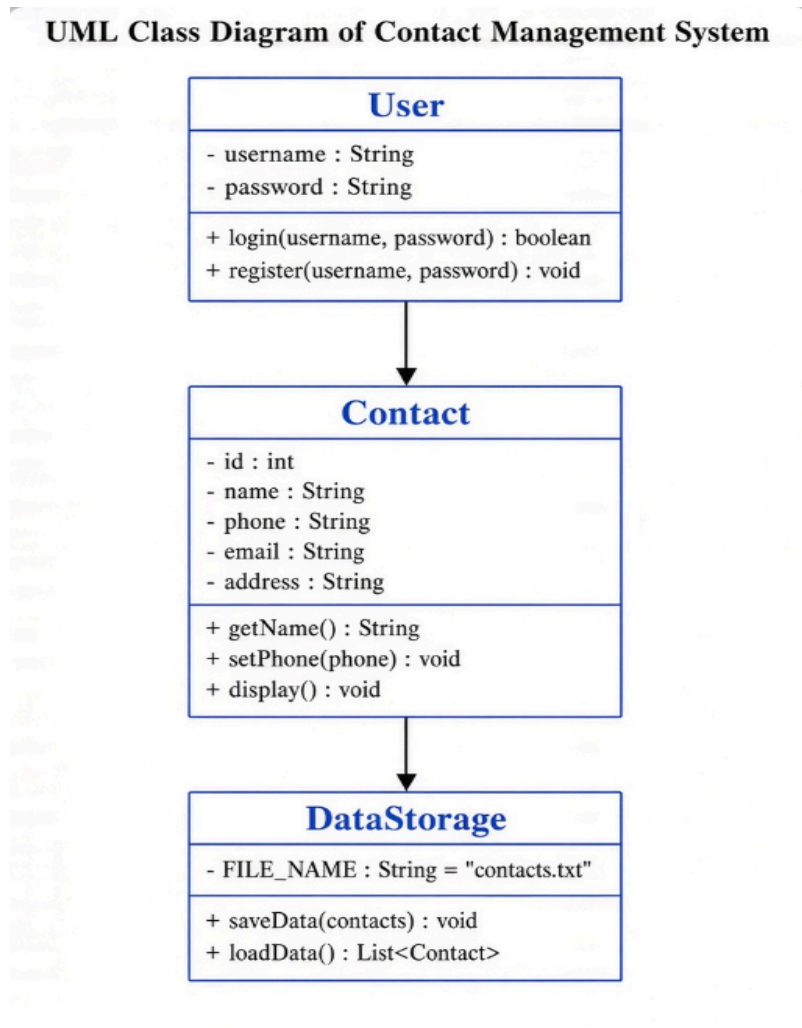


Figure 2.1: UML Class Diagram of Contact Management System

2.3.2 Class Order List

1. **User.java** – Stores user credentials and handles login/registration.
2. **Contact.java** – Holds individual contact data (Name, Phone, Email, Address).
3. **DataStorage.java** – Manages saving and reading data to/from text files.
4. **Main.java** – Main interface and menu controller.

2.3.3 Class Description

- **User**
Purpose: Stores user login information securely.
Methods: getters/setters, login validation, registration.
- **Contact**
Purpose: Contains contact details like name, phone number, email, and address.
Methods: getters, setters, display(), toString().
- **DataStorage**
Purpose: Reads and writes contact data to contacts.txt file.
Methods: saveContacts(), loadContacts().

- **2.3.4 Flowchart of Program Execution:**

-

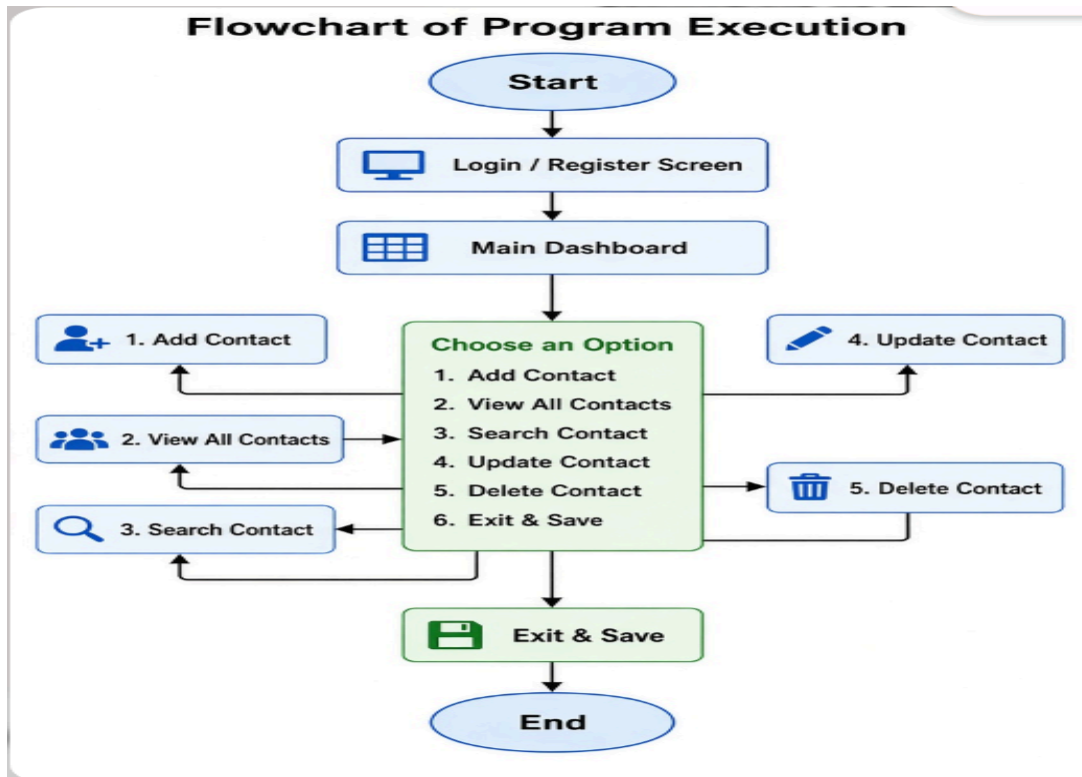


Figure 2.2: Flowchart of Program Execution

2.3.5 Dynamic Output Showcase with Code Insights :

Step 1: User Registration & Login

(Figure 2.3: User Registration & Login Output)

The system first asks the user to Register or Login. After successful authentication, the main menu is displayed.

Step 2: Adding a New Contact

(Figure 2.4: Adding New Contact Output)

User enters Name, Phone, Email, and Address. The contact is saved successfully.

Step 3: View, Search & Update Contact

(Figure 2.5: View & Update Contact Output)

Displays all contacts. User can search by name or phone and update any field.

Step 4: Delete Contact

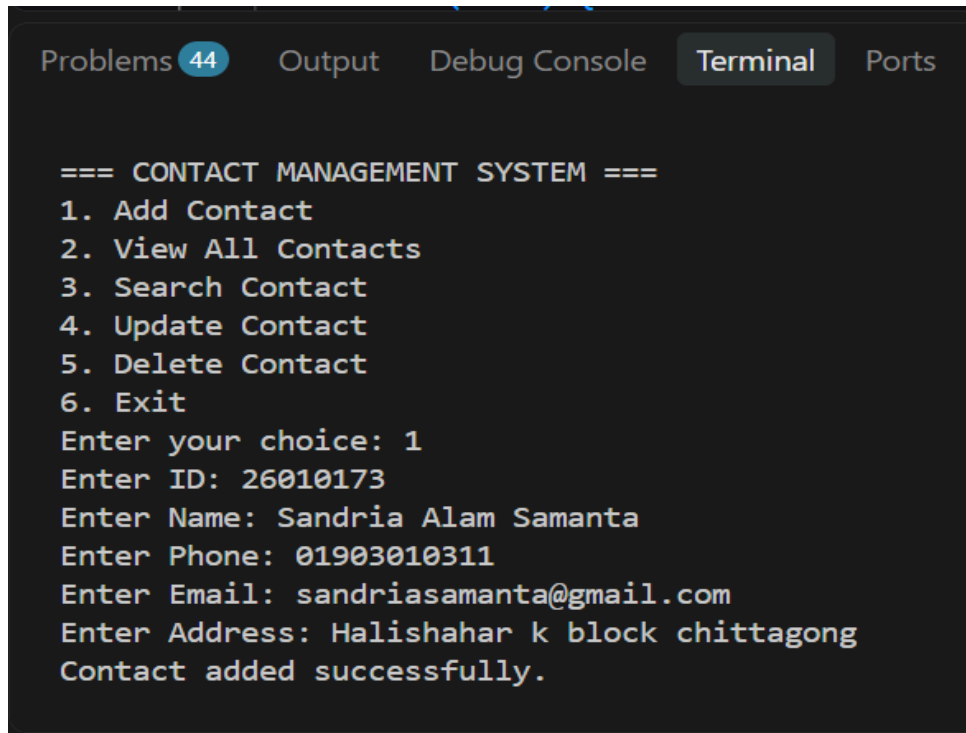
(Figure 2.6: Delete Contact)

Delete the Contact Successfully.

2.3.6. Execution Flow & Screenshots: This section contains the complete execution flow of the Contact Management System along with the terminal screenshots of all major features:

- User Registration and Login
- Adding a New Contact
- Viewing All Contacts
- Searching a Contact
- Updating a Contact
- Deleting a Contact
- Exiting and Saving Data

Figure 2.3(User Registration and Login):



```
Problems 44 Output Debug Console Terminal Ports

=== CONTACT MANAGEMENT SYSTEM ===
1. Add Contact
2. View All Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter your choice: 1
Enter ID: 26010173
Enter Name: Sandria Alam Samanta
Enter Phone: 01903010311
Enter Email: sandriasamanta@gmail.com
Enter Address: Haliashahar k block chittagong
Contact added successfully.
```

Explanation :

The main menu shows all the available options of the Contact Management System.

The user can add, view, search, update, or delete a contact from this menu. It provides a simple way to control the whole system.

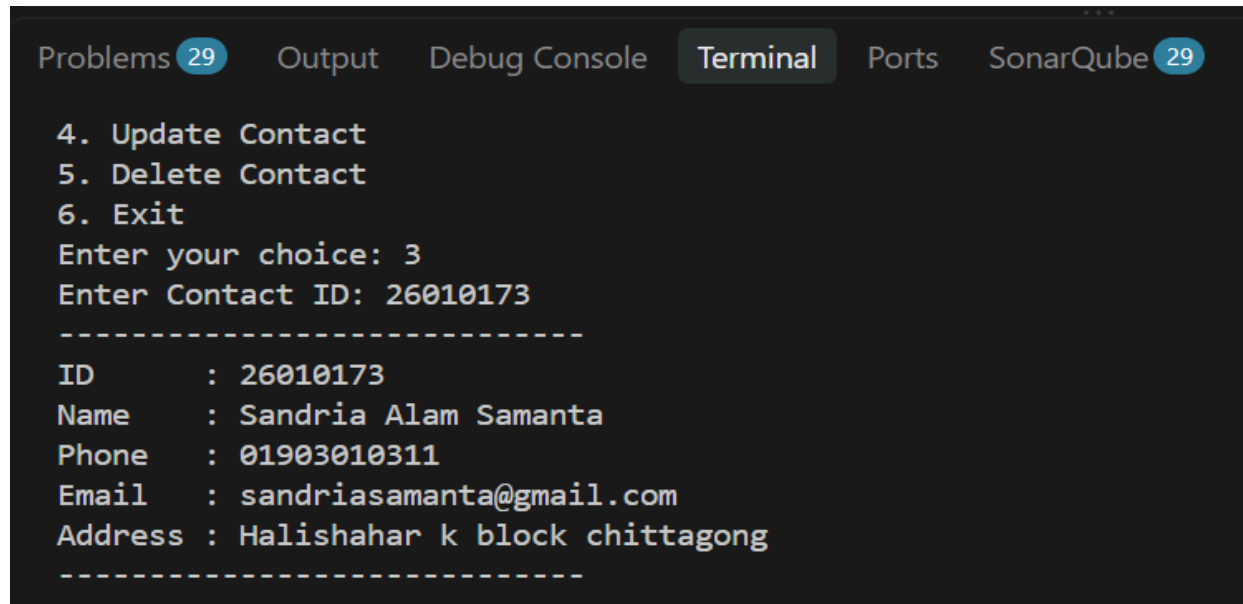
Figure 2.4 (Adding New Contact):

```
Enter your choice: 1
Enter ID: 26010173
Enter Name: Sandria Alam Samanta
Enter Phone: 01903010311
Enter Email: sandriasamanta@gmail.com
Enter Address: Haliashahar k block chittagong
Contact added successfully.
```

Explanation : This figure shows the process of adding a new contact to the system.

The user enters the contact ID, name, phone number, email, and address. After entering the information, the system shows a successful message.

Figure 2.5(Search Contact):

A screenshot of a terminal window with a dark background. At the top, there is a navigation bar with tabs: 'Problems' (with a blue circle containing '29'), 'Output', 'Debug Console', 'Terminal' (highlighted), 'Ports', and 'SonarQube' (with a blue circle containing '29'). The terminal content shows a menu with options: '4. Update Contact', '5. Delete Contact', and '6. Exit'. Below the menu, the user enters '3' for 'Update Contact', then '26010173' for the 'Contact ID'. A dashed line separates the input from the output. The output displays the contact details: 'ID : 26010173', 'Name : Sandria Alam Samanta', 'Phone : 01903010311', 'Email : sandriasamanta@gmail.com', and 'Address : Haliashahar k block chittagong'. Another dashed line follows the output.

```
Problems 29 Output Debug Console Terminal Ports SonarQube 29

4. Update Contact
5. Delete Contact
6. Exit
Enter your choice: 3
Enter Contact ID: 26010173
-----
ID      : 26010173
Name    : Sandria Alam Samanta
Phone   : 01903010311
Email   : sandriasamanta@gmail.com
Address : Haliashahar k block chittagong
-----
```

Explanation : This figure shows how a contact is searched using the contact ID. The system checks the entered ID and finds the matching contact. If the contact is found, its information is displayed on the screen.

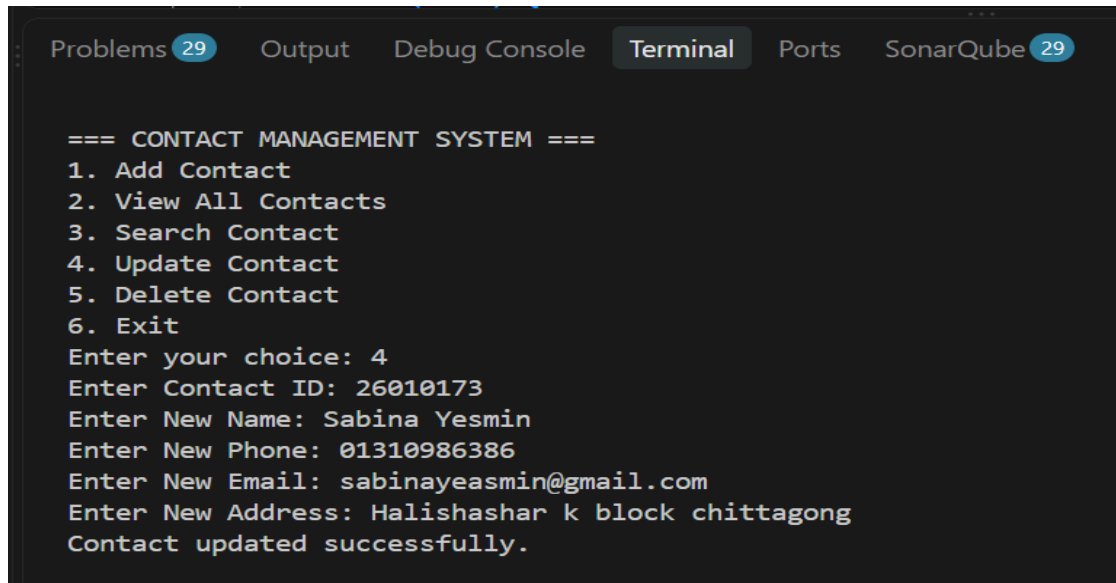
Figure 2.6 (View All Contact) :

```
Problems 29 Output Debug Console Terminal Ports SonarQube 29

=== CONTACT MANAGEMENT SYSTEM ===
1. Add Contact
2. View All Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter your choice: 2
-----
ID      : 26010173
Name    : Sandria Alam Samanta
Phone   : 01903010311
Email   : sandriasamanta@gmail.com
Address : Halishahar k block chittagong
-----
```

Explanation : This figure shows the saved contact information in the system. It displays the contact ID, name, phone number, email, and address. The user can easily view all saved contacts.

Figure 2.7(Update Contact) :



```
Problems 29 Output Debug Console Terminal Ports SonarQube 29

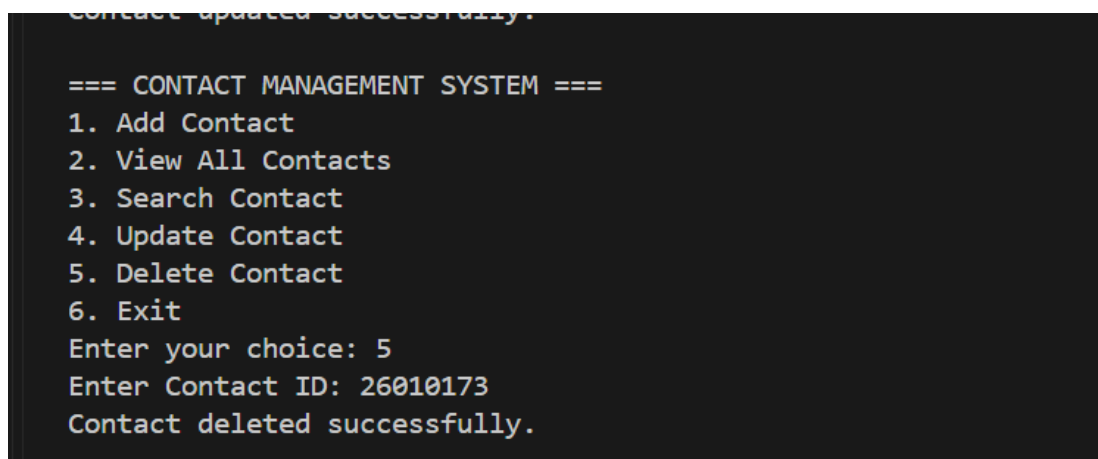
=== CONTACT MANAGEMENT SYSTEM ===
1. Add Contact
2. View All Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter your choice: 4
Enter Contact ID: 26010173
Enter New Name: Sabina Yesmin
Enter New Phone: 01310986386
Enter New Email: sabinayearmin@gmail.com
Enter New Address: Halishashar k block chittagong
Contact updated successfully.
```

Explanation : This figure shows how the information of an existing contact is updated.

The user enters the contact ID and provides the new information.

The system updates the contact and shows a successful message.

Figure 2.8 (Delete Contact) :



```
Contact updated successfully.

=== CONTACT MANAGEMENT SYSTEM ===
1. Add Contact
2. View All Contacts
3. Search Contact
4. Update Contact
5. Delete Contact
6. Exit
Enter your choice: 5
Enter Contact ID: 26010173
Contact deleted successfully.
```

Explanation: This figure shows how an existing contact is deleted from the system.

The user enters the contact ID of the contact to be deleted.
The system removes the contact and shows a successful message.

2.3.7 Key Technical Features:

- **User Authentication & Security:** Mandatory Login/Registration system.
- **File-Based Data Persistence:** Uses Java File I/O (contacts.txt) to save and load data.
- **Robust Exception Handling:** try-catch blocks for invalid inputs.
- **Modular OOP Structure:** Separation of Model (Contact/User), Controller (Main), and Data Layer (DataStorage).
- **OOP Principles Applied:** Encapsulation, Inheritance, Polymorphism, Abstraction, Method Overloading & Overriding.

2.3.8 How OOP Principles are Used:

- **Encapsulation:** Private fields in Contact and User classes with public getters & setters.
- **Inheritance:** Possible base class (e.g., Person) extended by Contact (code reuse & hierarchy).
- **Polymorphism:** Method overloading (different search methods) and method overriding (display methods).
- **Abstraction:** Hiding file handling and data storage details from the user.
- **Exception Handling:** Proper try-catch for file and input errors.
- **File Handling:** Reading and writing contacts using FileWriter / BufferedReader or Object Serialization.
- **Login/Registration:** Mandatory for all users.

2.3.9 Project Timeline (Phases with Estimated Time):

Phase	Description	Estimated Time	Status
Phase 1	Requirement Analysis & Console Interface Design	1 Week	Completed
Phase 2	OOP Class Design & User Authentication Logic	1 Week	Completed
Phase 3	File I/O Integration (contacts.txt)	1 Week	Completed
Phase 4	System Testing, Exception Handling & Documentation	1 Week	Completed

2.4 Conclusion:

This project successfully demonstrates the practical application of Object-Oriented Programming (OOP) principles in Java by developing a fully functional **Contact Management System**. The system allows users to securely register and log in, add new contacts, view all contacts, search by name or phone number, update existing contact details, and delete contacts when required. All data is stored permanently using File Handling, so no information is lost even after closing the application.

Through this project, key OOP concepts such as Encapsulation, Abstraction, Inheritance, Polymorphism, Method Overloading, Method Overriding, and Exception Handling have been applied in a real-life scenario. The modular design

using separate classes (User, Contact, DataStorage, and Main) makes the code clean, reusable, and easy to maintain.

Overall, the Contact Management System fulfills its main goal of providing a lightweight, reliable, and user-friendly console-based application for managing personal or professional contacts efficiently.

2.4.1 Remaining Work:

Although the core features of the system are complete and working properly, the following improvements can still be made:

- Adding phone number format validation (so that only valid numbers can be entered).
- Implementing password hashing for better security instead of storing plain text passwords.
- Checking for duplicate contacts before adding a new one.
- Adding a feature to sort contacts alphabetically by name.
- Improving the search feature to support partial matching more accurately.
- Adding confirmation messages before deleting a contact to avoid accidental deletion.

2.4.2 Future Work:

In the future, this project can be upgraded in the following ways:

- Developing a Graphical User Interface (GUI) using JavaFX or Swing to make it more user-friendly.
- Integrating a database (MySQL or SQLite) instead of using text files for better data management.
- Adding contact categories such as Friends, Family, Work, and Others.
- Implementing an option to export contacts to CSV or Excel format.
- Adding a backup and restore feature for contact data.
-
- Creating a mobile version of the application using Android.

2.5 . Reference List :

<https://www.youtube.com/watch?v=Pg1i5Nw7lbM>

<https://www.scribd.com/document/927440862/Java-Microproject-Report>

1. Class Lectures and Lab Materials of CSE124 – Object Oriented Programming.

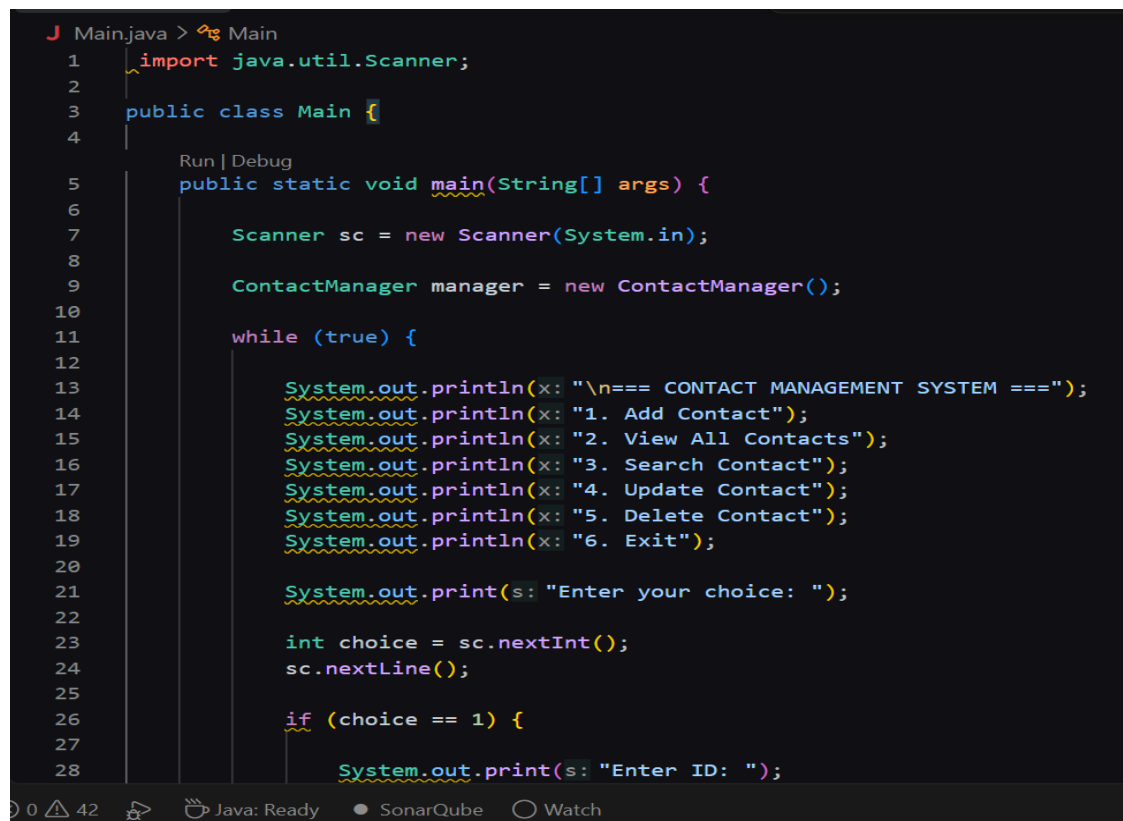
2.6 Appendix:

2.6.1 Github Link: <https://github.com/samantasandria-debug/Contact-Management-System>

2.6.2 Code Snippets:

This section contains the important code parts of the project for better understanding.

Figure: Main.java .



```
J Main.java > Main
1  import java.util.Scanner;
2
3  public class Main {
4
5      Run | Debug
6      public static void main(String[] args) {
7
8          Scanner sc = new Scanner(System.in);
9
10         ContactManager manager = new ContactManager();
11
12         while (true) {
13
14             System.out.println(x: "\n=== CONTACT MANAGEMENT SYSTEM ===");
15             System.out.println(x: "1. Add Contact");
16             System.out.println(x: "2. View All Contacts");
17             System.out.println(x: "3. Search Contact");
18             System.out.println(x: "4. Update Contact");
19             System.out.println(x: "5. Delete Contact");
20             System.out.println(x: "6. Exit");
21
22             System.out.print(s: "Enter your choice: ");
23
24             int choice = sc.nextInt();
25             sc.nextLine();
26
27             if (choice == 1) {
28                 System.out.print(s: "Enter ID: ");
```

```

20
21     System.out.print(s: "Enter your choice: ");
22
23     int choice = sc.nextInt();
24     sc.nextLine();
25
26     if (choice == 1) {
27
28         System.out.print(s: "Enter ID: ");
29         int id = sc.nextInt();
30         sc.nextLine();
31
32         System.out.print(s: "Enter Name: ");
33         String name = sc.nextLine();
34
35         System.out.print(s: "Enter Phone: ");
36         String phone = sc.nextLine();
37
38         System.out.print(s: "Enter Email: ");
39         String email = sc.nextLine();
40
41         System.out.print(s: "Enter Address: ");
42         String address = sc.nextLine();
43
44         Contact contact =
45             new Contact(id, name, phone, email, address);
46

```

```

J Main.java > Main
3     public class Main {
5         public static void main(String[] args) {
47             manager.addContact(contact);
48
49         }
50
51         else if (choice == 2) {
52
53             manager.viewContacts();
54
55         }
56

```

```
55     }
56
57     else if (choice == 3) {
58
59         System.out.print(s: "Enter Contact ID: ");
60
61         int id = sc.nextInt();
62
63         Contact contact = manager.findContact(id);
64
65         if (contact != null) {
66
67             contact.display();
68
69         } else {
70
71             System.out.println(x: "Contact not found.");
72         }
73     }
74
75     else if (choice == 4) {
76
77         System.out.print(s: "Enter Contact ID: ");
78
79         int id = sc.nextInt();
80         sc.nextLine();
81     }
```

```
81
82     Contact contact = manager.findContact(id);
83
84     if (contact != null) {
85
86         System.out.print(s: "Enter New Name: ");
87         contact.setName(sc.nextLine());
88
89         System.out.print(s: "Enter New Phone: ");
90         contact.setPhone(sc.nextLine());
91
92         System.out.print(s: "Enter New Email: ");
93         contact.setEmail(sc.nextLine());
94
95         System.out.print(s: "Enter New Address: ");
96         contact.setAddress(sc.nextLine());
97
98         System.out.println(
99             x: "Contact updated successfully."
100         );
101
102     } else {
103
104         System.out.println(x: "Contact not found.");
105     }
106 }
107
```

```

107
108     else if (choice == 5) {
109
110         System.out.print(s: "Enter Contact ID: ");
111
112         int id = sc.nextInt();
113
114         manager.deleteContact(id);
115     }
116
117     else if (choice == 6) {
118
119         System.out.println(x: "Thank you for using Contact Management System.");
120
121         break;
122     }
123
124     else {
125
126         System.out.println(x: "Invalid choice.");
127     }
128 }
129
130 sc.close();
131 }
132

```

Discussion : This code is the main part of the Contact Management System. It provides a menu where the user can add, view, search, update, and delete contacts. The Scanner class is used to take information and choices from the user.

Figure: Contact Manager.java:

```
J ContactManager.java > ...
1  import java.util.ArrayList;
2
3  public class ContactManager {
4
5      private ArrayList<Contact> contacts = new ArrayList<>();
6
7      public void addContact(Contact contact) {
8
9          contacts.add(contact);
10
11          System.out.println(x: "Contact added successfully.");
12      }
13
14      public void viewContacts() {
15
16          if (contacts.isEmpty()) {
17
18              System.out.println(x: "No contacts found.");
19
20              return;
21          }
22
23          for (Contact contact : contacts) {
24
25              contact.display();
26          }
27      }
28
29      public Contact findContact(int id) {
```

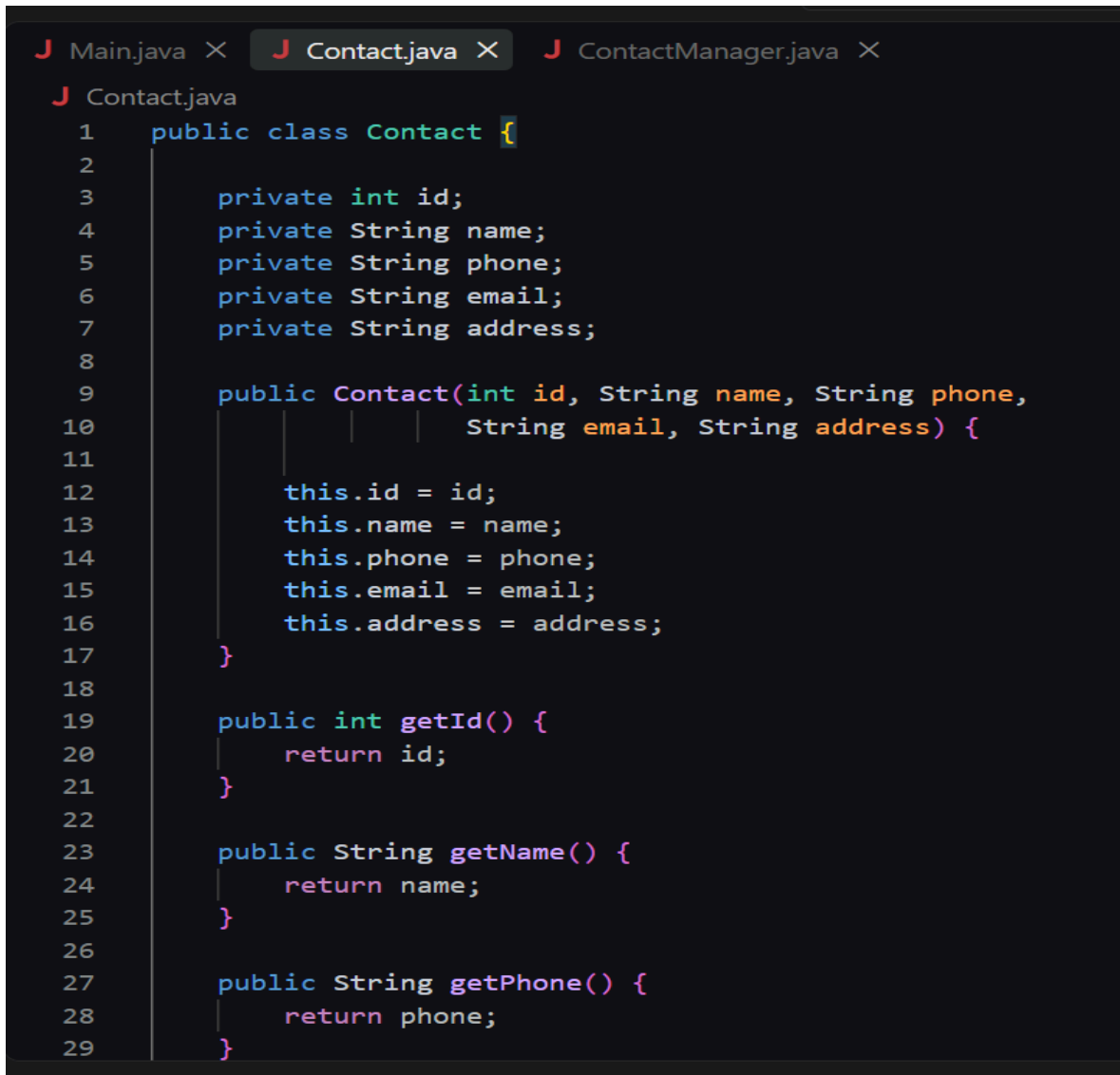
0 42 Java: Ready SonarQube Watch

```
32
33          if (contact.getId() == id) {
34
35              return contact;
36          }
37      }
38
39      return null;
40  }
41
42  public void deleteContact(int id) {
43
44      Contact contact = findContact(id);
45
46      if (contact != null) {
47
48          contacts.remove(contact);
49
50          System.out.println(x: "Contact deleted successfully.");
51      } else {
52
53          System.out.println(x: "Contact not found.");
54      }
55  }
56
57  }
58
```

0 42 Java: Ready SonarQube Watch

Discussion: This code manages all the contacts using an ArrayList. It provides methods to add, view, search, and delete contacts. It helps the main program organize and manage the stored contact information.

Figure: Contact .java :



```
J Main.java X J Contact.java X J ContactManager.java X
J Contact.java
1  public class Contact {
2
3      private int id;
4      private String name;
5      private String phone;
6      private String email;
7      private String address;
8
9      public Contact(int id, String name, String phone,
10                     String email, String address) {
11
12          this.id = id;
13          this.name = name;
14          this.phone = phone;
15          this.email = email;
16          this.address = address;
17      }
18
19      public int getId() {
20          return id;
21      }
22
23      public String getName() {
24          return name;
25      }
26
27      public String getPhone() {
28          return phone;
29      }
```

```
J Main.java X J Contact.java 8 X J ContactManager.java 5 X
J Contact.java
30
31     public String getEmail() {
32         return email;
33     }
34
35     public String getAddress() {
36         return address;
37     }
38
39     public void setName(String name) {
40         this.name = name;
41     }
42
43     public void setPhone(String phone) {
44         this.phone = phone;
45     }
46
47     public void setEmail(String email) {
48         this.email = email;
49     }
50
51     public void setAddress(String address) {
52         this.address = address;
53     }
54
55     public void display() {
56
57         System.out.println(x: "-----");
58         System.out.println("ID      : " + id);
```

```
J Main.java X J Contact.java 8 X J ContactManager.java 5 X
J Contact.java
56
57         System.out.println(x: "-----");
58         System.out.println("ID      : " + id);
59         System.out.println("Name   : " + name);
60         System.out.println("Phone  : " + phone);
61         System.out.println("Email  : " + email);
62         System.out.println("Address : " + address);
63         System.out.println(x: "-----");
64     }
65 }
```

Discussion: This code creates the Contact class to store contact information. It contains the contact ID, name, phone number, email, and address. Getter and setter methods are used to access and update the contact information.

Problems Faced & New Learnings

Problems Faced:

- Handling file read and write exceptions properly without crashing the program.
- Managing empty contact list when no data is available.
- Validating user input so that the program does not break on wrong entries.
- Keeping the code modular and avoiding repetition while using multiple classes.
- Formatting the output neatly in the console so that it looks clean and professional.

New Learnings:

- Practical use of Encapsulation by making variables private and using getters and setters.
- How to implement File Handling in Java using BufferedReader and BufferedWriter.
- Proper use of Exception Handling with try-catch blocks.
- Designing a complete system using multiple classes and connecting them properly.
- Understanding the importance of modular programming and clean code structure.
- How to create a simple authentication system (Login & Registration) in a console application

Summary of the Project:

The Contact Management System is a console-based Java application developed using Object-Oriented Programming concepts. It provides a complete solution for managing personal contacts with features like registration, login, adding, viewing, searching, updating, and deleting contacts. Data is stored permanently using File Handling so that users do not lose their information. The project successfully applies Encapsulation, Abstraction, Inheritance, Polymorphism, Exception Handling, and File Handling in a practical way. This system can be useful for students, teachers, and professionals who want a simple offline contact organizer without depending on any internet connection or cloud service.